E03 2016 memo - The memo for EO3 of the 2016 November Exam. The exam covers Recursion, Linked

Program Design: Introduction (University of Pretoria)

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Faculty of EBIT
Department of Computer Science
Program Design: Introduction / Progammeringsontwerp: Inleiding
COS 110

Exam Opportunity 3 / Eksamen Geleentheid 3
22/11/2016

Surname, Initials:_____

Student/Staff Nr:_____

Interne Eksaminatore / Internal Examiners:
Dr M Helbig, Prof AP Engelbrecht, Ms A Rakitianskaia, Ms N Banda, Mr. O. Babajide and Mr KS Ka

Eksterne Eksaminator / External Examiner:
Mr M Riekert

Instruksies / *Instructions*

1. This is an openbook event. You may make use of any paper-based material. / *Hierdie is 'n oopboek geleentheid. Jy mag enige papiergebaseerde materiaal gebruik.*
2. You have 180 minutes to complete this question paper. / *Jy het 180 minute om die vraestel te voltooi.*
3. There are 82 marks available in this paper. / *Daar is 82 punte beskikbaar in die vraestel.*
4. Programmable devices may not be used. / *Programmeerbare toerusting mag nie gebruik word nie.*
5. Answer all the questions. Give answers in the space provided. / *Beantwoord al die vrae. Gee antwoorde in die ruimte wat daarvoor verskaf word.*
6. Indicate any rough work clearly. / *Dui enige rofwerk duidelik aan.*
7. No answers in pencil will be marked. Only answer in pen. / *Geen antwoorde in potlood sal gemerk word nie. Antwoord alle vrae slegs in pen.*

Marks:

Question:	1	2	3	4	**Total**
Points:	18	9	30	25	**82**
Score:					

Question 1 Afdeling A / *Section A* ..... (18 marks)

Vir elkeen van die kodesegmente wat hieronder gegee word, identifiseer alle foute wat jy raaksien. Verduidelik wat die fout is, en hoe dit reggestel moet word. Let op dat punte afgetrek sal word indien jy korrekte kode as foutief aandui. Vir al die vrae hieronder, neem aan dat die volgende stellings ingesluit is: /
For each of the code segments given below, identify all the errors that you see. Explain what the errors are, and how they should be corrected. Take note that marks will be deducted if you indicated correct code as being in error. For all the questions below, assume that the following statements have been included:

```
#include <iostream>
using namespace std;
```

1.1 Beskou die volgende `makefile`: / (2)
Consider the following makefile:

```
FLAGS = -Wall -static
CC = g++

Node.o: Node.C Node.h
        $(CC) $(FLAGS) -c Node.C
```

waar die `Node` klas as volg gedefinieer is: /
where the `Node` class is defined as follows:

```
template <typename T>
class Node {
   T* info;
   Node* next;

   public:
      Node(void);
      Node(T*,Node*=0);
      ~Node(void);
};
```

Solution: Cannot compile template class to object code

1.2 Vir die volgende kodesegment, aanvaar dat alle lid funksies gespesifiseer en korrek geimplementeer is, en aanvaar ook dat die `IntSLLNode` klas gedefinieer en korrek geimplementeer is. / (4)
For the following code segment, assume that all the member functions are specified and correctly implemented, and also assume that the class `IntSLLNode` is defined and correctly implemented.

```
class IntSLList {
   private:
      IntSLLNode *head, *tail;

   public:
      IntSLList(void);
      ~IntSLList(void);
      //other constructors and member functions
      int deleteFromHead(void);
};
```

```
int IntSLList::deleteFromHead(void) {
   int el = head->info;
   IntSLLNode *tmp = head;
   if (head != tail)
      head = head->next;
   else
      head = tail = 0;
   delete tmp;
   return el;
}
```

Solution: Give full marks for any two of:

1. First line of function should test if head != 0 (or head == 0, or NULL)
2. Set tmp− >next = 0 before delete tmp
3. Return statement if head == NULL

1.3 Beskou die volgende gedeeltelike klas definisie: / (6)
Consider the following partial class definition:

```
template <typename T>
class DLLNode {
   private:
      T* element;
      DLLNode* next;
      DLLNode* previous;

   public:
      //constructors and destructors as necessary
      DLLNode& operator=(const DLLNode&);
      //other member functions
};

template <typename T>
DLLNode& DLLNode::operator=(const DLLNode& dN) {
   if (element)
      delete element;
   element = new DLLNode(dN.element,dN.next,dN.previous);
   return *this;
}
```

Solution: Give full marks for any three of:

1. Implementation of operator= should refer to $DLLNode < T >$
2. First test if this == &dN (or this != &dN)
3. Does not make sense to assign an new node to element
4. Update next and prev

1.4 (6)

```
template <typename T>
class Node {
   private:
      T* datum;
      Node* next;

   public:
      Node(void);
      Node(const T& _datum, Node* _next = 0);
      Node(const Node&);
      ~Node(void);
};

template <typename T>
class List {
   private:
      Node<T> *head;

   public:
      List(void);
      List(const Node<T>&);
      List(const List&);
      insert(const Node<T>&);
      remove(const T&);
      ~List(void);
};

int main(void) {
   List<int> list1;
   //code to insert elements into list1
   List<int> list2;
   //code to insert elements into list2
   list1 = list2;
   cout << list1 << endl;
}
```

Solution: Give full marks for any 3 of:

1. No operator= overloaded for Node
2. `insert(const Node<T>&)` should be `insert(const T&)`
3. No operator= overloaded for List / shallow copy at list1 = list2;
4. No operator<< for List

Question 2 Afdeling B / *Section B* ..... (9 marks)

Vir elkeen van die volgende kode fragmente, wat sal vertoon word? /
For each of the following code fragments, what will be displayed?

2.1
```
void someRecursion(int i) {
    if (i > 0) {
       cout << i << ' ';
       someRecursion(i-1);
    }
}
```

a) Vir parameter waarde 0. / (1)
For parameter value 0.

Solution: Nothing

b) Vir parameter waarde 4. / (2)
For parameter value 4.

Solution: 4 3 2 1

2.2
```
void someRecursion(int i) {
    if (i > 0) {
       someRecursion(i-1);
       cout << i << ' ';
    }
}
```

a) Vir parameter waarde 4. / (2)
For parameter value 4.

Solution: 1 2 3 4

2.3
```
void someRecursion(int i) {
    if (i != 0) {
       someRecursion(i-2);
       cout << i << ' ';
       someRecursion(i-1);
    }
}
```

a) Vir parameter waarde 4. / (2)
For parameter value 4.

Solution: 2 1 4 1 3 2 1
OR:
Infinite recursion
(condition must be changed to $i > 0$ to stop infinite recursion)

b) Vir parameter waarde 3. / (2)
For parameter value 3.

Solution: 1 3 2 1
OR:
Infinite recursion
(condition must be changed to $i > 0$ to stop infinite recursion)

Question 3 Afdeling C / *Section C* ..... (30 marks)

Skryf **rekursiewe** funksies om die volgende probleme op te los. Let op dat geen punte gegee sal word indien oplossings nie rekursief is nie. /

Write **recursive** *functions to solve each of the following problems. Note that no marks will be given if solutions are not recursive.*

3.1 Skep en implimenteer 'n rekursiewe funksie `int ack(int m, int n)` vir die Ackermann funksie wat as volg gedefinieer is: / (6)

Create and implement a recursive function `ack(int m, int n)` *for the Ackermann function which is defined as follows:*

```
A(m,n) = n + 1 if m == 0
A(m,n) = A(m-1, 1) if m > 0 && n == 0
A(m,n) =A(m-1, A(m, n-1)) if m > 0 && n > 0
```

Solution:

```
int ack(int m, int n) {
    if(m == 0)                          \\ 1 mark
         return n + 1;                  \\ 1 mark
    else if (m > 0 && n == 0)           \\ 1 mark
         return ack(m-1, 1);            \\ 1 mark
    else if(m > 0 && n > 0)             \\ 1 mark
         return ack(m-1, ack(m,n-1));\\ 1 mark
    else
        throw ''Inputs must be positive''; \\ not necessary!
}
```

3.2 Skep en implimenteer 'n rekursiewe funksie `int mc(int m)` vir die McCarthy 91 funksie wat as volg gedefinieer is: / (4)

Create and implement a recursive function `mc(int m)` *for the McCarthy 91 function which is defined as follows:*

```
M(n) = n - 10 if n> 100
M(n) = M(M(n+11)) if n <= 100
```

Solution:

```
int mc(int n) {
    if(n > 100)                         \\ 1 mark
         return n - 10;                 \\ 1 mark
    else
         return mc(mc(n + 11));         \\ 2 marks
}
```

3.3 Skryf 'n rekursiewe funksie, `ordered`, om te toets of 'n skikking van wisselpunt getalle in dalende orde voorkom of nie. Die funksie stuur `true` terug indien die getalle in dalende orde is, en `false` indien nie. / (4)
Write a recursive function, `ordered`, to test if an array of floating-point numbers is given in decreasing order. The function returns `true` if the numbers are in decreasing order, and `false` if not.

Solution:

```
bool ordered(double * arr, int size, int index = 0) {
    if(index == size)
         return true;
    if(arr[index] < arr[index + 1])
         return false;
    return ordered(arr, size, index + 1);
}
```

This one can vary - make sure student's **recursive** logic makes sense, and if it does, award marks

3.4 Skryf 'n rekursiewe funksie, `recCos`, om die volgende reeks wat die funksie $\cos(x)$ benader, te implementeer: / (6)
Write a recursive function, `recCos`, to implement the following series to approximate the function $\cos(x)$*:*

$$\cos(x) = \sum_{k=0}^{t-1} (-1)^k \frac{x^{2k}}{(2k)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Die funksie ontvang as parameters die waarde van x en die aantal terme wat gebruik moet word in die benadering. Aanvaar dat die funksie `unsigned int factorial(unsigned int n)`, wat $n!$ bereken, reeds bestaan. /
The function receives as parameters the value of x and the number of terms that should be used in the approximation. Assume that the function `unsigned int factorial(unsigned int n)`, which calculates $n!$, already exists.

Solution:

```
#include <math>

unsigned int recCos(int x, unsigned int t) {
    if(t <= 1)                          // 1 mark
         return 1;                      // 1 mark
    else
         return pow(-1,t-1) *           // 1 mark
                pow(x,2*(t-1)) /        // 1 mark
                factorial(2 * (t-1)) +  // 1 mark
                recCos(x, t-1);         // 1 mark
}
```

This one can vary - make sure student's **recursive** logic makes sense, and if it does, award marks

3.5 Beskou die volgende definisie van 'n enkel geskakelde lys: /
Consider the following definition of a singly linked list:

```
class SLLNode {
   int info;
   SLLNode *next;

   public:
      //constructors and destructors
};

class SLList {
   private:
      SLList *head;

   public:
      //constructors and destructors
      //member functions
};
```

a) Skryf 'n rekursiewe funksie, `int frequency(int item)`, wat die aantal voorkomste van die parameter in die lys tel en terug stuur. / (6)
Write a recursive function, `int frequency(int item)`, to count and to return the number of occurrences of the parameter in the list.

Solution:

```
int SLList<T>::frequency(int item) {
    return freq(head, item);
}

int SLList<T>::freq(SLLNode * n, item) {
    if(n == NULL)                       // 1 mark
        return 0;                       // 1 mark
    if(n->info == item)                 // 1 mark
        return 1 + freq(n->next, item);// 1 mark
    else
        return freq(n->next, item);     // 2 marks
}
```

This one can vary - make sure student's **recursive** logic makes sense, and if it does, award marks

b) Skryf 'n rekursiewe funksie, `void reverse(void)`, om die inhoud van die lys in omgekeerde orde te vertoon. / (4)
Write a recursive function, `void reverse(void)`, to display the contents of list in reverse order.

Solution:

```
void SLList<T>::reverse(void) {
    reverse(head);                      // 1 mark
}

void SLList<T>::reverse(SLLNode * n) {
    reverse(n->next);                   // 1 mark
    if(n != NULL)                       // 1 mark
        cout << n-> info << endl;       // 1 mark
    // Can also have:
    // if (n == NULL) return;           // 1 mark
}
```

This one can vary - make sure student's **recursive** logic makes sense, and if it does, award marks

Question 4 Afdeling D / *Section D* ..... (25 marks)

Vir die geskakelde lys definisie in Afdeling C, beantwoord die volgende vrae: /
For the linked list definition in Section C, answer the following questions:

4.1 Skryf die versuim konstrueerder vir die `SLLNode` klas. / (6)
Write the default constructor for the `SLLNode` class.

Solution:

```
SLLNode::SLLNode() { // 2 marks
    next = NULL;     // 2 marks
    info = 0;        // 2 marks
}
```

4.2 Skryf die dekonstrueerder vir die `SLList` klas. / (4)
Write the destructor for the `SLList` class.

Solution:

```
SLList::~SLList() {
    SLLNode * ptr = head;
    while(head != NULL)    // 1 mark
    {
        head = head->next; // 1 mark
        delete ptr;        // 1 mark
        ptr = head;        // 1 mark
    }
}
```

4.3 Oorlaai die indeks operator sodat die volgende stelling geldig is: / (15)
Overload the index operator so that the following statement can be valid:

```
SLList *myList;

for (int i = 1; i <= 10; i++)
   myList[i-1] = i;

myList[5] = 100;
myList[0] = 0;
```

Solution:

```
int& SLList::operator[](int index) {        // 1 mark for correct return type
    int counter = 0;
    SLLNode * ptr = head;
    SLLNode * prev = 0;
    while(counter < index && ptr != NULL)       // 2 marks for correct loop
    {
        counter++;                              // 1 mark
        prev = ptr;                             // 1 mark
        ptr = ptr->next;                        // 1 mark
    }
    if(ptr != NULL)
        return ptr->info;                       // 2 marks: if not null, return
    else {
        SLLNode * nNode = new SLLNode;          // 1 mark: create new node
        if(prev == NULL)
                head = nNode;                   // 2 marks: update head
        else
                prev->next = nNode;             // 2 marks: attach new node
        return  nNode->info;                    // 2 marks: return new node
    }
}
```